

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**

ИНСТИТУТ ИНТЕЛЛЕКТУАЛЬНОЙ РОБОТОТЕХНИКИ

Авторы:

Долгих Алексей Александрович (25945)
Здори́кова Ксения Андреевна (25945)
Старцева Виктория Павловна (25945)

Название работы:

«Сапёр»

Новосибирск, 2026

1 Введение	3
2 Аппаратная часть.....	4
3 Программная часть.....	13
4 Заключение	19

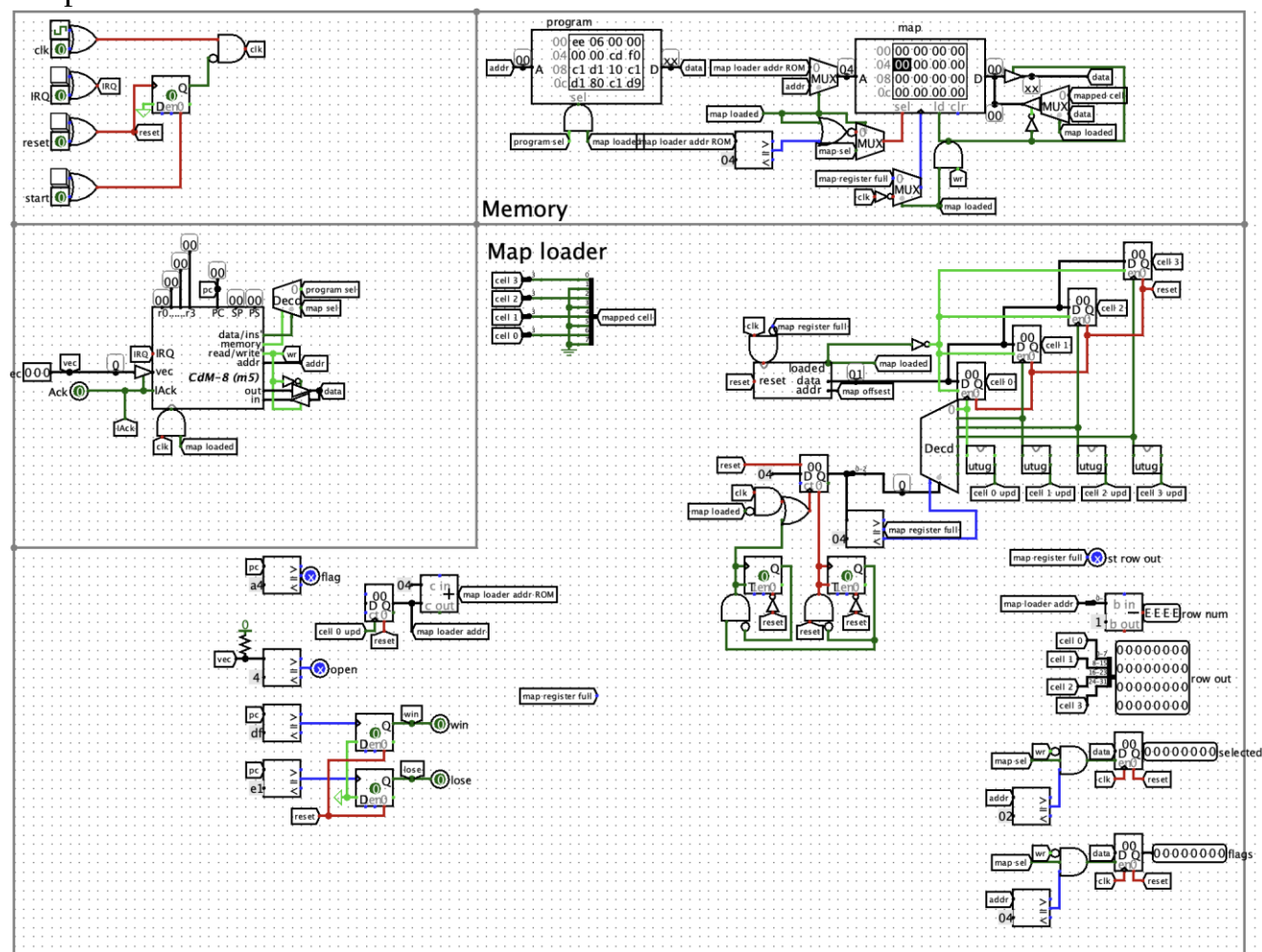
1 Введение

В качестве курсового проекта по предмету «Цифровые платформы» мы решили выбрать игру «Сапёр» и реализовали её с помощью полученных знаний во время обучения. Нашими главными инструментами были Logisim и процессора CDM8-mark5.

Сапер – это компьютерная игра, в которой пользователю нужно очистить минное поле, открывая безопасные клетки. Свою популярность она получила за счет включения в состав операционной системы Microsoft Windows. Изначально игрок открывает любую ячейку и на поле появляются цифры, обозначающие количество мин вокруг них. Далее человеку нужно открывать, те клетки, которые не являются минами. Если ячейка является плохой, то ставиться флажок.

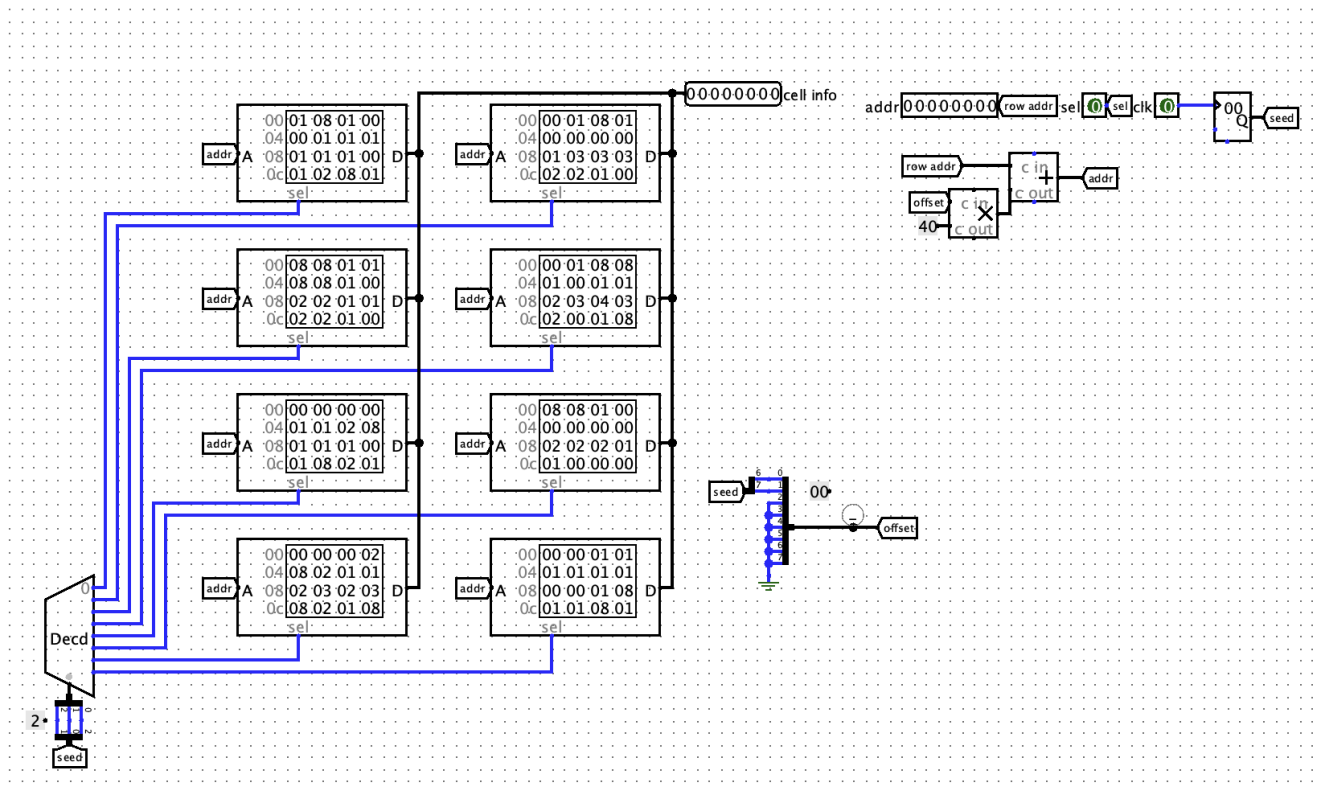
2 Аппаратная часть

2.1 processor handler



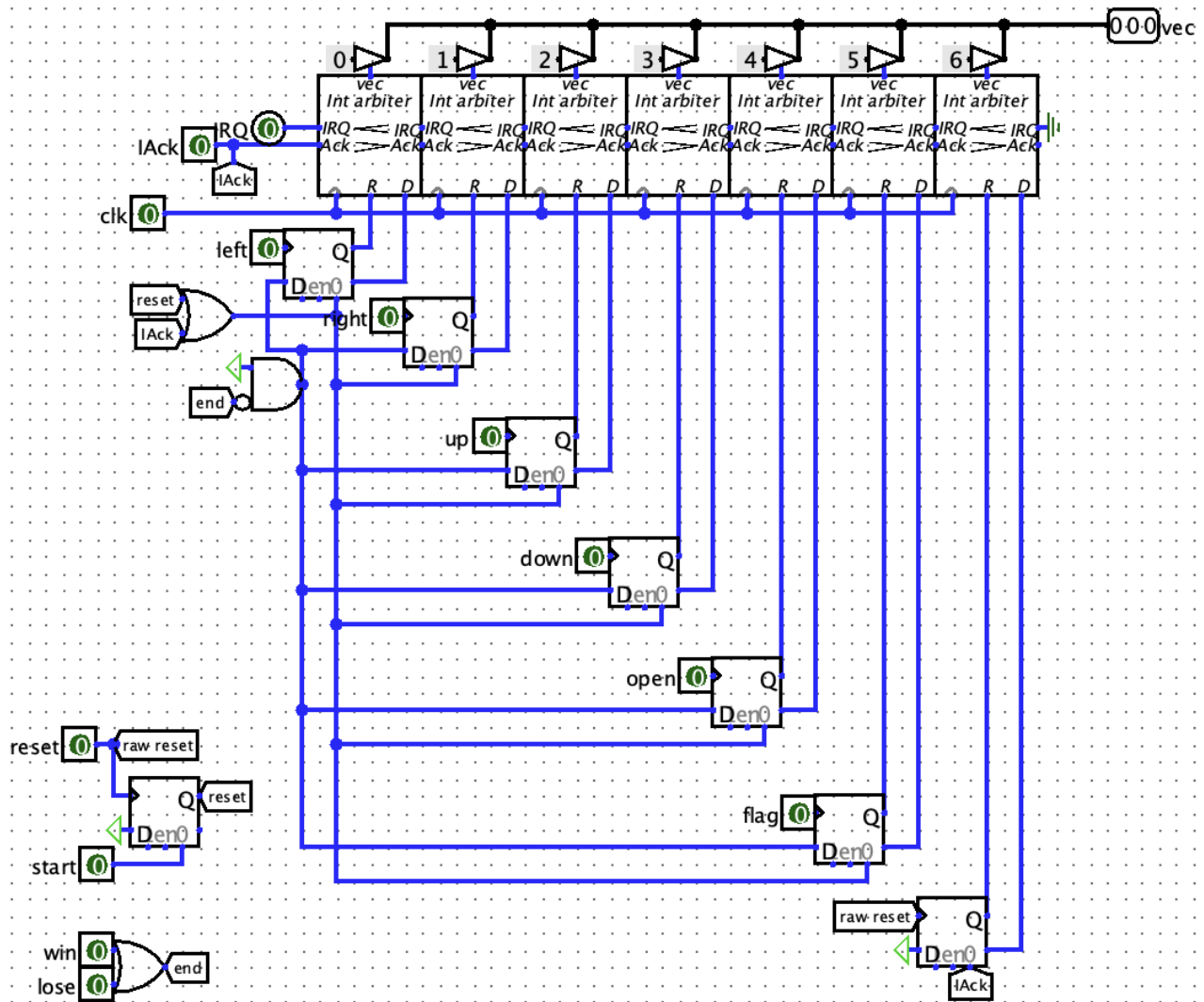
Модуль processor handler является центральным вычислительным ядром игры «Сапёр» в проекте CdM-8. Он обрабатывает прерывания от кнопок, управляет загрузкой карты уровней из ROM, хранит состояние игрового поля в RAM, проверяет условия победы или поражения и генерирует 32-битные данные row out для отрисовки строк на дисплее. Внутри модуля находятся ROM с картой мин, RAM состояния, регистры текущей ячейки, счётчик флагов, модуль загрузки карты map_loader и логика рекурсивного открытия пустых областей. При нажатии «О» выполняется проверка ячейки: если мина — выставляется сигнал lose, если пусто — открываются соседи, если цифра — отображается число. При нажатии «F» устанавливается или снимается флаг и обновляется выход flags. Победа наступает, когда все мины отмечены и все безопасные ячейки открыты. Модуль синхронизирован тактовым сигналом, взаимодействует с контроллером ввода, видеоконтроллером и модулями отрисовки, обеспечивая полную игровую логику на цифровых элементах без использования программного кода.

2.2 map_selector



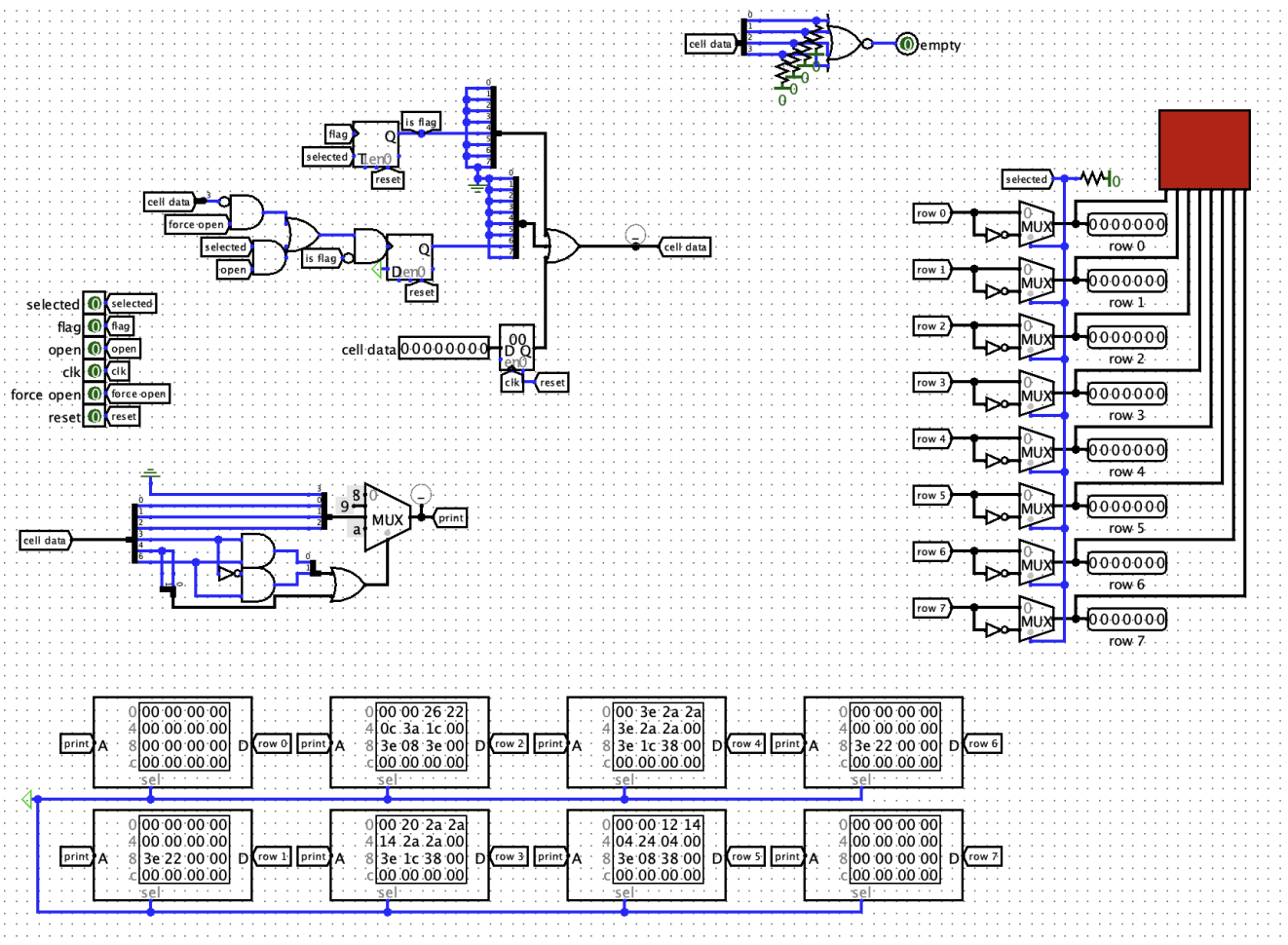
Модуль `map_selector` отвечает за преобразование координат выбранной ячейки в адрес ПЗУ с картой уровней и возврат значения этой ячейки, которое указывает, содержит ячейка мину или нет. Внутри модуля находятся четыре ПЗУ объёмом 256 байт каждое, которые хранят predetermined игровые уровни. Входы модуля: `addr`, `sel`, `clk`. Выход `cell info` содержит данные о содержимом ячейки. Также есть дополнительный выход `seed`, который используется для генерации случайных чисел. Алгоритм работы следующий: значение `addr` умножается на константу `0x40` в умножителе, затем складывается с `sel` в сумматоре, полученный результат поступает на адресные входы всех четырёх ПЗУ, а их выходы через декодер под управлением `sel` передаются на выход `cell info`. Таким образом модуль поддерживает до восьми предустановленных уровней сложности. Дополнительно присутствует генератор случайных чисел, который может использоваться для создания случайных раскладов мин, но в данной версии, используются фиксированные карты. Модуль синхронизирован тактовым сигналом и выставляет на выходе `cell info` байт, где каждый бит может означать наличие мины или другие параметры ячейки.

2.3 controller



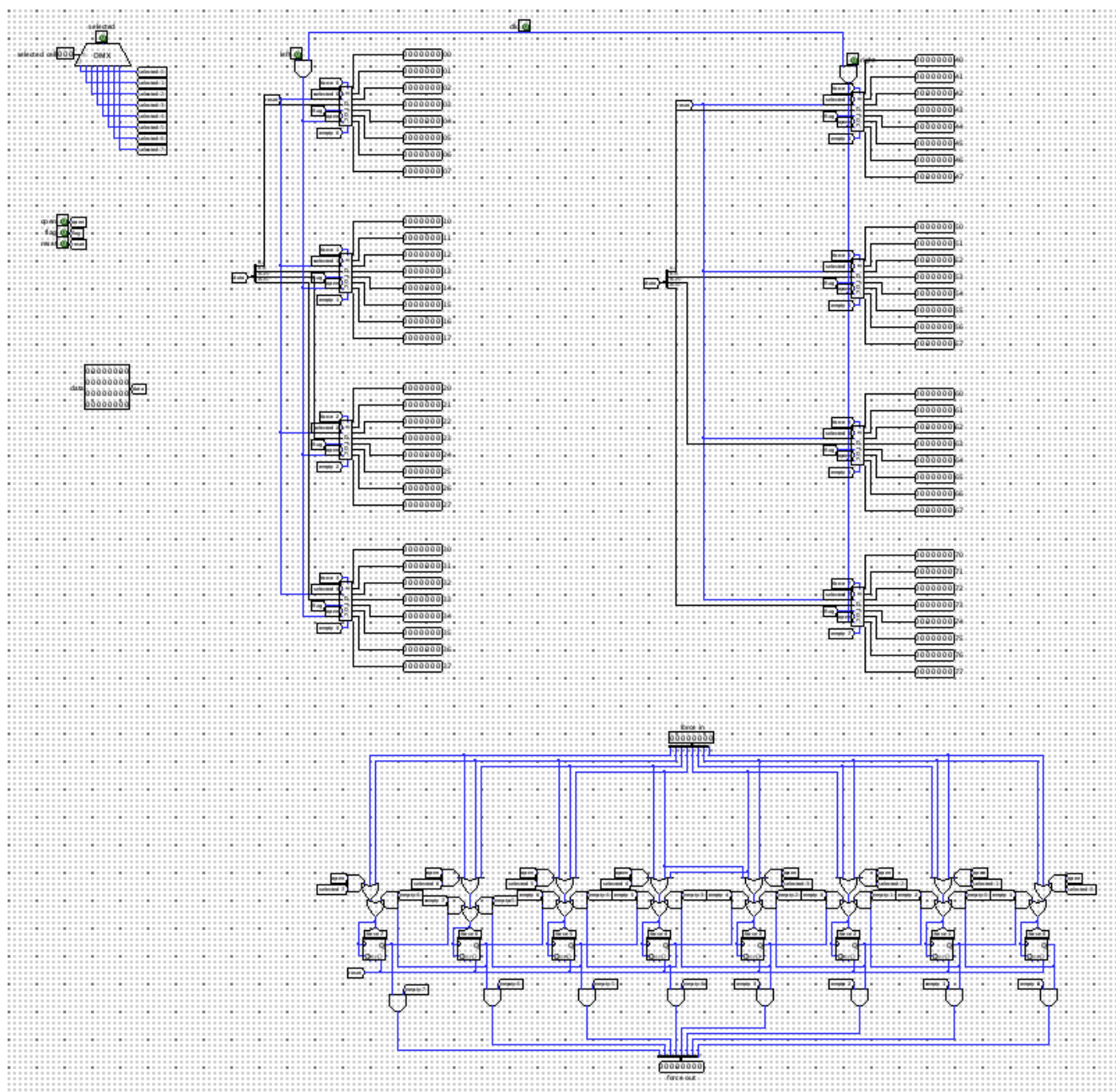
Модуль controller обрабатывает нажатия кнопок up, down, left, right, open, flag, а также сигналы reset, start, win, lose и clk. На выходе он формирует 3-битный вектор vec, кодирующий действие игрока, и сигналы IRQ и IAck. Внутри находятся арбитры для разрешения конфликтов одновременных нажатий, D-триггеры для запоминания нажатий, декодеры и управляемые буферы. Сигналы reset и start управляют состоянием игры, а win и lose блокируют обработку нажатий после окончания игры. Выходной вектор vec поступает в processor handler для выполнения действия.

2.4 cell_drawer



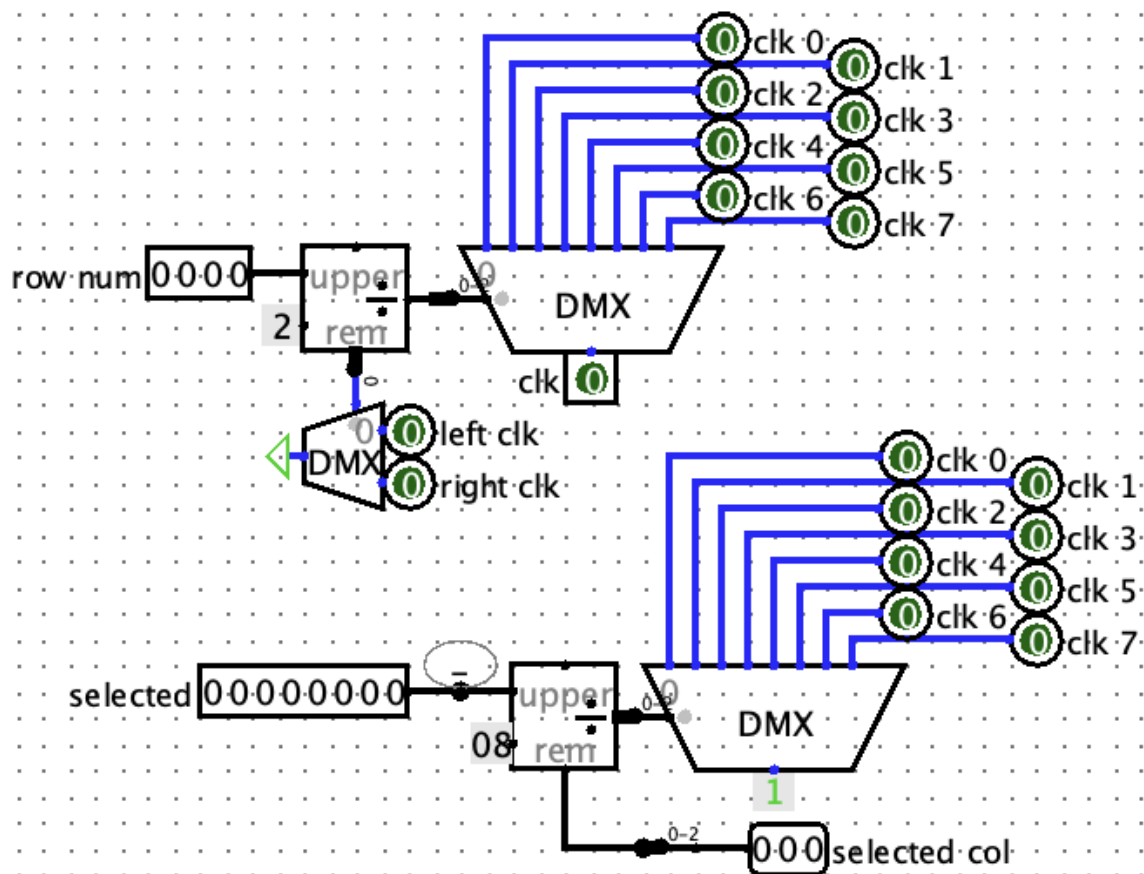
Модуль `cell_drawer` рисует одну ячейку 7x8 пикселей. На вход он получает тип ячейки, сигналы выбора, флага, открытия, `clk` и `reset`. Внутри находятся ПЗУ с образами цифр, мин, флага, пустой и закрытой клеток. Мультиплексоры выбирают нужный образ в зависимости от состояния ячейки. Сигнал `selected` подсвечивает выбранную ячейку, `force open` принудительно открывает её. На выходе формируются 8 сигналов по 7 бит каждый для строк пикселей. Выходные данные передаются в `row_drawer` для сборки строки.

2.5 row_drawer



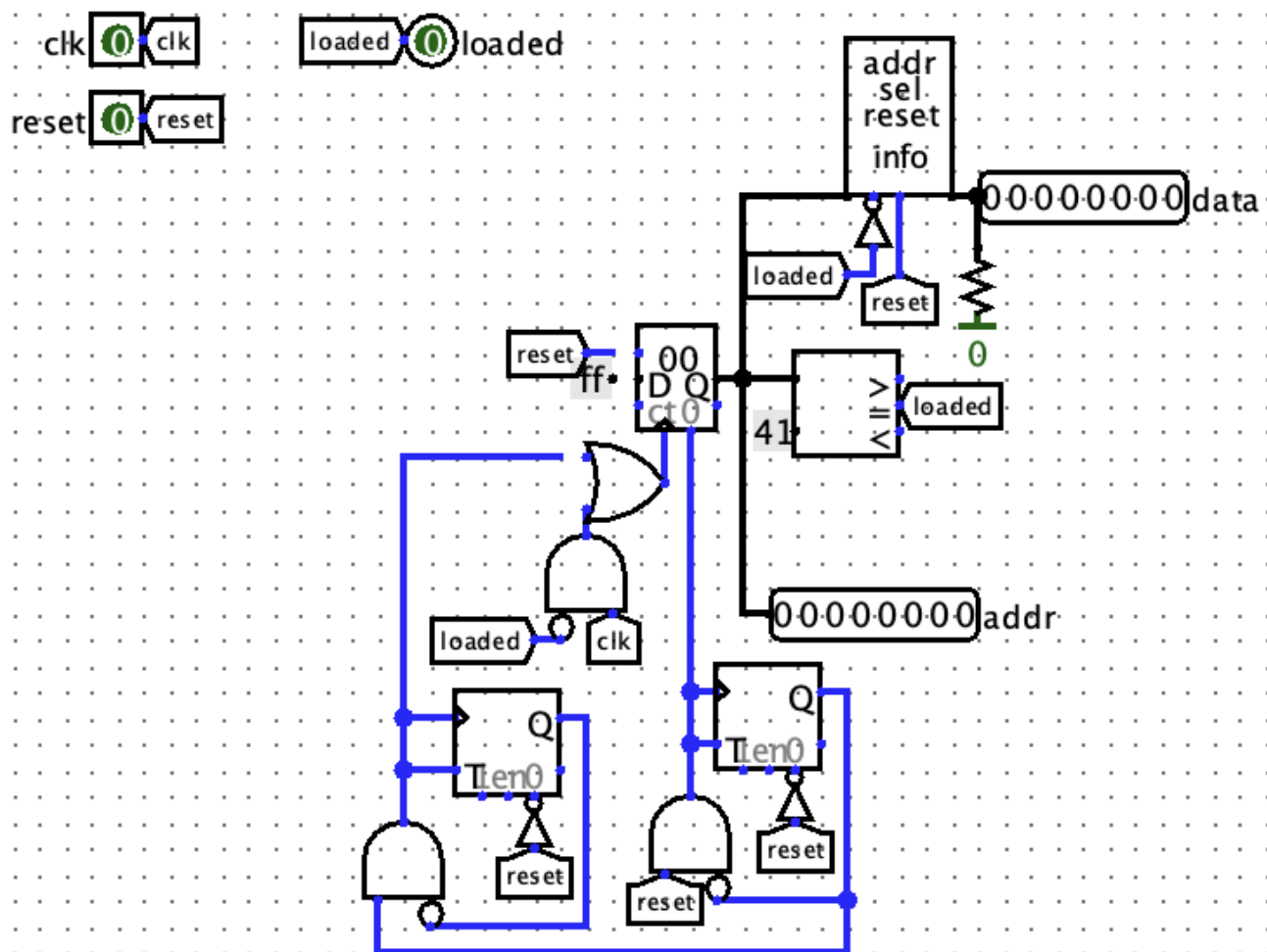
Модуль `row_drawer` отвечает за отрисовку одной строки игрового поля из восьми ячеек. Внутри него находятся восемь копий схемы `cell_drawer`, каждая из которых рисует одну ячейку. На вход модуль получает 32-битные данные о всей строке, сигналы выбора для каждой ячейки, а также сигналы `open`, `flag`, `force` и `reset`. Эти данные распределяются между восемью `cell_drawer`, которые формируют изображение каждой ячейки. Все выходы объединяются в готовую строку из 64 пикселей и отправляются на дисплей. Сигнал `force` принудительно открывает ячейки при проигрыше для показа всех мин.

2.6 video handler



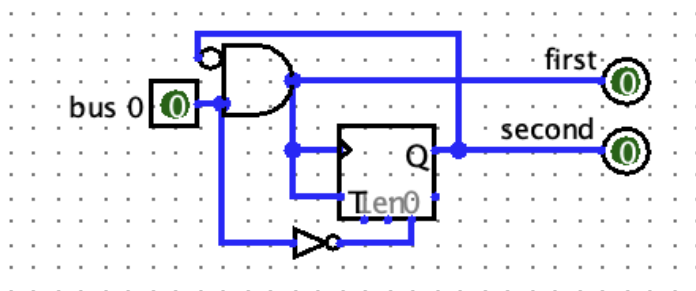
Модуль video handler отвечает за генерацию тактовых сигналов для синхронного обновления строк игрового поля. На вход он получает общий тактовый сигнал clk и 3-битный номер выбранной строки, а на выходе формирует восемь независимых тактовых сигналов clk0...clk7, а также отдельные сигналы для левой и правой половин экрана. Внутри модуля находятся делители частоты и дешифраторы, которые преобразуют входной номер строки в соответствующий выходной тактовый сигнал. Это позволяет активировать только ту строку, которая в данный момент должна быть обновлена на дисплее. Модуль также содержит компаратор и дополнительные делители, которые обеспечивают равномерное распределение тактов между строками. Таким образом video handler управляет вертикальной развёрткой дисплея, последовательно запуская отрисовку строк 0, 1, 2 и так далее, что создаёт устойчивое изображение на матрице светодиодов.

2.7 map_loader



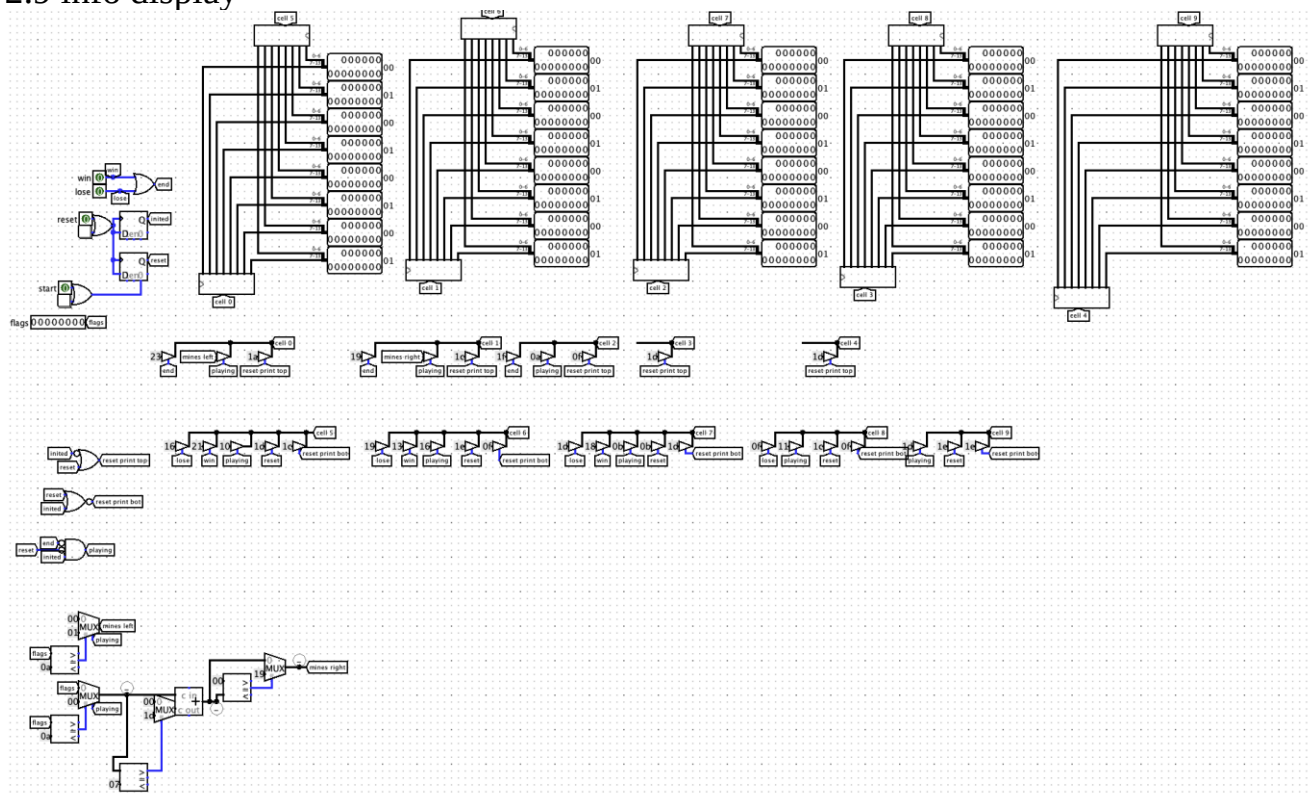
Модуль `map_loader` отвечает за автоматическую загрузку карты уровней из ПЗУ в игровое поле при старте новой игры. На вход он получает тактовый сигнал `clk` и сигнал `reset`, а на выходе формирует 8-битные адрес `addr` и данные `data`, а также сигнал `loaded`, который указывает на завершение загрузки. Внутри модуля находятся счётчик, который последовательно перебирает все 64 адреса ячеек поля, компаратор, проверяющий достижение конечного адреса, и несколько триггеров, управляющих процессом. При подаче сигнала `reset` счётчик обнуляется и начинается цикл загрузки: на каждом такте `clk` счётчик увеличивается, формируется очередной адрес, данные из ПЗУ по этому адресу записываются в игровое поле. Когда счётчик доходит до 64, компаратор активирует сигнал `loaded`, который останавливает процесс загрузки. Таким образом `map_loader` автоматически заполняет всё поле минами и пустыми клетками по заданному шаблону без участия основной игровой логики.

2.8 delayer



Модуль delayer представляет собой простую схему задержки сигнала на один такт. На вход он получает сигнал bus 0, а на выходе формирует два сигнала: first и second. Внутри модуля находятся Т-триггер и инвертор. Сигнал first повторяет входной сигнал с минимальной задержкой, а сигнал second представляет собой инвертированную и задержанную копию входного сигнала. Такая схема используется для синхронизации операций чтения и записи в память, когда необходимо разнести во времени фазы выборки адреса и обмена данными. Также delayer может применяться для формирования стробирующих импульсов из фронтов тактового сигнала или для создания двухфазных сигналов управления.

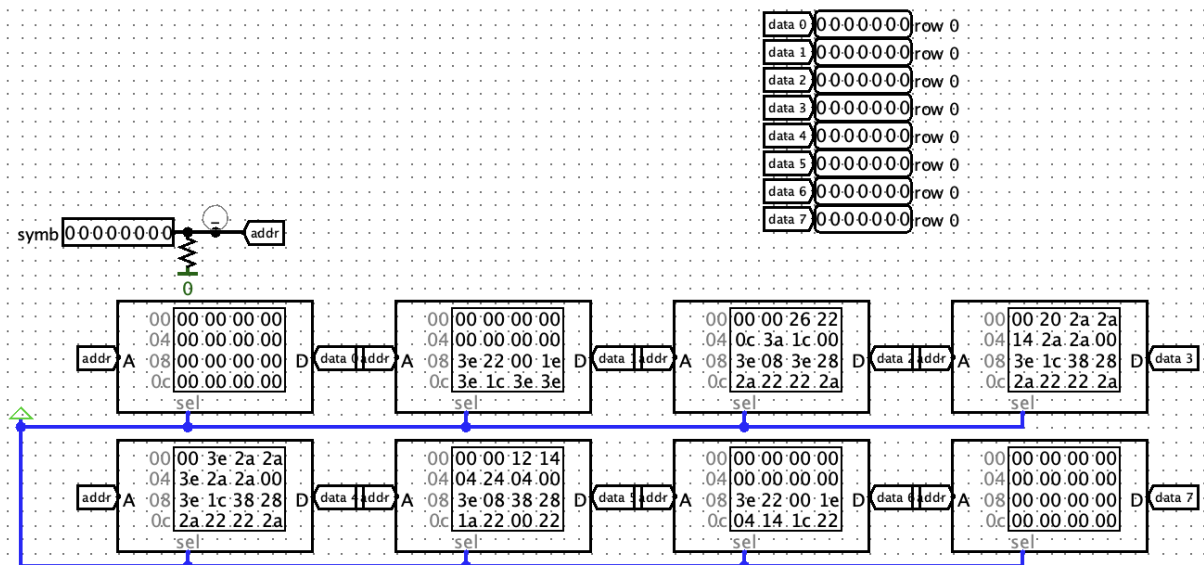
2.9 info display



Модуль info display отвечает за отображение текстовой информации на экране: количества оставшихся мин, сообщений о победе или поражении, а также других игровых параметров. На вход он получает 8-битные значения клеток cell 0...cell 9, сигналы win, lose, reset, а также тактовый сигнал clk. Внутри модуля находятся несколько контролируемых буферов, которые под управлением сигналов playing и reset print top, reset print bot выбирают нужный образ символа из ПЗУ. Модуль использует символьный генератор для преобразования числовых значений в пиксельные образы цифр и букв. Выходные данные модуля организованы в виде четырёх 14-битных шин, каждая из которых управляет одним знакоместом на

дисплее. При победе модуль выводит слово win, при поражении lose, а в процессе игры отображает счётчик оставшихся мин. Сигнал сброса очищает экран и возвращает модуль в начальное состояние.

2.10 symb_drawer



Модуль `symb_drawer` отвечает за отрисовку одного символа на дисплее. На вход он получает 8-битный код символа, который поступает на адресные входы внутренних ПЗУ. Внутри модуля находятся четыре ПЗУ, каждое из которых хранит образы символов размером 7 на 8 пикселей. Первое ПЗУ содержит образы цифр и базовых символов, второе и третье хранят дополнительные символы, а четвёртое предназначено для специальных игровых обозначений. Выходы всех четырёх ПЗУ объединены через шины и подаются на выходные порты `row0...row7`, каждый из которых представляет собой 7-битный сигнал для одной строки пикселей символа. Таким образом модуль преобразует входной код символа в готовое растровое изображение, которое затем отображается на матрице. Модуль работает асинхронно, без тактового сигнала, так как ПЗУ являются комбинационными схемами.

3 Программная часть

Программная часть реализована на CDM8-mark4.

3.1 Входные данные

```
asect 0x00
br main

cursor: ds 1
mines: ds 1
flags: ds 1
map: ds 1
```

asect 0x00 – начало абсолютной секции в памяти. Программа начнет выполняться с адреса 0x00. Далее у нас выполняется переход к подпрограмме main.

Так же у нас есть четыре основных переменных:

cursor – текущая позиция курсора

mines – оставшееся количество мин

flags – оставшееся количество флажков

map – сжатая карта

3.2 Основной цикл и обработка прерываний

```
main:
    setsp 0xf0

    ei
    jsr reset
main_loop:
    wait
    br main_loop
```

В подпрограмме main мы устанавливаем указатель стека на начало области стека с помощью setsp. Потом разрешаем прерывания и вызываем подпрограмму инициализации игры.

main_loop останавливает выполнение до следующего прерывания.

3.3 Движение курсора

У нас есть четыре подпрограммы left, right, up и down, которые вызываются при нажатии клавиш.

```
left:
    ldi r2, cursor
    ld r2, r0

    if
        ldi r1, 0x07
        and r0, r1
    is z
        ldi r1, 0x07
        add r1, r0
    else
        dec r0
    fi

    st r2, r0

    rti
```

Сначала мы загружаем текущую позицию курсора в r0. Далее с помощью маски выделяем первые 3 бита, которые отвечают за координату X. Если результат операции and равен нулю, это означает, что курсор находится в крайнем левом столбце (X=0). В этом случае к текущей позиции прибавляется 7, что перемещает курсор в крайний правый столбец той же строки. Если же курсор не находится на границе, то выполняется уменьшение значения на 1, следовательно, сдвигая курсор влево. После перемещения мы сохраняем новую позицию, и выполняется инструкция rti, которая возвращает управление прерванной программе. Подпрограммы right, up и down имеют похожую структуру, поэтому они не будут описываться.

3.4 Декодирование адреса и масок клетки

```
# r0 - chunk data
# r1 - cell num
# r2 - chunk addr
set_cells_data:
    ldi r2, cursor
    ld r2, r1
    push r1

    shra r1
    shra r1

    ldi r2, map
    add r1, r2

    ld r2, r0

    pop r1

    rts
```

Подпрограмма set_cells_data предназначена для загрузки в регистр r0 байта карты, содержащего информацию о четырёх игровых клетках, одной из которых является текущая позиция курсора. Сначала подпрограмма загружает адрес переменной cursor в r2, читает значение курсора из памяти в r1 и сохраняет его в стек с помощью push. Затем значение курсора дважды сдвигается вправо, что является целочисленным делением на 4. Поскольку каждая клетка кодируется двумя битами, а в одном байте хранятся данные о четырёх последовательных клетках, результатом деления номера клетки на 4 является номер байта в карте, содержащего нужную группу. Далее в r2 загружается базовый адрес карты, к нему прибавляется вычисленное смещение, и по полученному адресу из памяти читается байт, который сохраняется в r0. В конце подпрограмма восстанавливает из стека исходное значение r1 и возвращает управление через rts.

```

#args:
#r1 - cell num

#result
#r0 - corrupted
#r2 - flag mask
#r3 - mine mask
get_cell_masks:
    ldi r0, 0x03

    ldi r2, 0x80
    ldi r3, 0x40

    and r1, r0

    while
        tst r0
        stays ne
        shr r3
        shr r3

        shr r2
        shr r2

        dec r0
    wend

    rts

```

Подпрограмма `get_cell_masks` принимает в `r1` номер клетки и возвращает в `r2` битовую маску флага, а в `r3` — битовую маску мины для этой клетки. Сначала в `r0` загружается константа `0x03`, которая используется для выделения младших двух битов номера клетки. Эти два бита определяют позицию клетки внутри байта карты, поскольку один байт кодирует четыре клетки по два бита на каждую. В `r2` и `r3` загружаются начальные маски: `0x80` для флага и `0x40` для мины, соответствующие самой левой клетке в байте. Затем запускается цикл `while`, который выполняется столько раз, сколько указано в младших двух битах `r0`. На каждой итерации маски сдвигаются вправо на два бита, что позволяет перейти к маске следующей клетки внутри байта. После завершения цикла в `r2` и `r3` находятся маски, соответствующие нужной клетке. Подпрограмма завершается инструкцией `rts`.

3.5 Открытие ячейки


```

open:
    jsr set_cells_data
    push r0
    jsr get_cell_masks
    pop r0

    if
        and r0, r2
    is ne
        rti
    fi

    if
        and r0, r3
    is ne
        br lose
    fi

    rti

```

Подпрограмма open является обработчиком прерывания для действия «открыть клетку» в игре «Сапёр». Она отвечает за проверку содержимого клетки, на которой в данный момент находится курсор, и определяет, приведёт ли это действие к продолжению игры, победе или поражению. Изначально загружается байт карты в r0, потом сохраняется в стек. Далее мы получаем информацию о маске флага и мин, загружая их в r2 и r3 соответственно. Если на клетке есть флаг, то ничего не происходит, просто выходим. Если на клетке есть мина, то проигрывш.

3.6 Установка и снятие флага

```

flag:
    jsr set_cells_data
    push r2 #addr
    push r0 #data
    jsr get_cell_masks
    #r2 - flag
    #r3 - mine
    ldi r1, flags
    ld r1, r1

    if
        tst r1
        is z
        pop r0
        push r0
        and r2, r0
        is z
    then
        pop r0
        pop r0
        rti
    fi

    pop r0
    xor r2, r0
    pop r1
    st r1, r0

    if
        and r0, r2
    is z #remove flag
        ldi r1, 0x01
        if
            and r0, r3
            is ne #is mine
            ldi r2, 0x01
        else
            ldi r2, 0x00
        fi
    else #put flag
        ldi r1, 0xff
        if
            and r0, r3
            is ne #is mine
            ldi r2, 0xff
        else
            ldi r2, 0x00
        fi
    fi

    ldi r3, mines
    ld r3, r0
    add r2, r0

    if
        tst r0
        is z
        br win
    fi

```



```

ldi r3, mines
ld r3, r0
add r2, r0

if
    tst r0
is z
    br win
fi

st r3, r0

inc r3
ld r3, r0
add r1, r0
st r3, r0

rti

```

Подпрограмма `flag` является обработчиком прерывания для установки и снятия флажков. При её вызове сначала с помощью `set_cells_data` загружается байт карты, содержащий информацию о четырёх клетках, включая текущую позицию курсора, а затем этот байт сохраняется в стеке. Вызов `get_cell_masks` вычисляет битовые маски флага и мины для нужной клетки. После этого подпрограмма проверяет, есть ли у игрока доступные флажки: если флажков нет и на клетке уже нет флага, операция немедленно прерывается через `rti`, так как установить новый флаг невозможно. Далее из стека восстанавливается байт карты, и с помощью операции `xor` с маской флага бит флага переключается на противоположное значение — если флага не было, он устанавливается, если был — снимается. Обновлённый байт сохраняется обратно в память. Затем подпрограмма определяет, был ли флаг установлен или снят, и была ли эта клетка миной, после чего соответствующим образом изменяет счётчики: `mines` и `flags`. Если после обновления счётчик `mines` становится равным нулю, это означает, что все мины правильно помечены флажками, и подпрограмма переходит к метке `win` для обработки победы. В противном случае выполнение завершается инструкцией `rti`, возвращая управление прерванной программе.

3.7 Установка значения мин и флагов

```

reset:
    ldi r0, mines
    ldi r1, 0x0A
    st r0, r1
    inc r0
    st r0, r1
    rts

```

Подпрограмма `reset` устанавливает начальные значения счетчиков мин и флагов. Изначально мы загружаем адрес переменной `mines` в `r0`. Потом загружается значение 10 в `r1` и сохраняем количество мин в переменную `mines`. После этого регистр `r0` увеличивается на единицу (`inc r0`), чтобы он указывал на следующую переменную — `flags`, которая хранит количество оставшихся у игрока флажков. То же самое значение 10 сохраняется и в эту переменную, устанавливая начальный запас флажков также равным 10.

3.8 Инициализация новой игры

```
start:
    jsr reset
    ldi r0, cursor
    ldi r1, 0x00
    st r0, r1
    rti
    ldi r1, 0x00
```

Первым делом выполняется подпрограмма `reset`. После этого подпрограмма `start` загружает в регистр `r0` адрес переменной `cursor`, которая хранит текущую позицию курсора на игровом поле. В регистр `r1` загружается значение `0`, после чего инструкция `st r0, r1` сохраняет этот ноль в переменную `cursor`, устанавливая курсор в самое начало поля — в верхний левый угол.

3.9 Выигрыш и проигрыш

```
win: br main
lose: br main
```

Данные подпрограммы являются обработчиками завершения игры при победе или поражении соответственно и выполняют одну функцию — переход к `main`.

3.10 Таблица векторов прерываний в конце памяти

```
asect 0xf0
dc left, 0x00      #0x00
dc right, 0x00     #0x01
dc up, 0x00        #0x02
dc down, 0x00      #0x03
dc open, 0x00      #0x04
dc flag, 0x00      #0x05
dc start, 0x00     #0x06

end.
```

Данный фрагмент кода определяет таблицу векторов прерываний в памяти процессора CdM-8. Директива `asect 0xf0` указывает ассемблеру разместить следующие данные начиная с абсолютного адреса `0xf0`.

4 Заключение

В ходе реализации проекта была реализована игра сапёр с использованием Logisim и процессора CDM8.